Statoo — Cahier des charges

Version 1.1 | Cible V1 (APK Android - Flutter) | Schéma DB v1

1. Contexte et objectif

Statoo est une application personnelle de suivi d'un groupe de foot 5x5 entre amis. Elle est conçue pour un seul utilisateur : l'administrateur du groupe, qui saisit les séances depuis son téléphone. L'application est installée sur un seul appareil. Il n'y a aucun compte, ni aucun partage en temps réel.

> **Objectif :** Remplacer le fichier tableur et les notes du téléphone par un outil sur-mesure qui enregistre une séance en moins de deux minutes et produit un classement fiable.
>
> **Non-objectif :** Servir de terrain d'expérimentation architectural. L'approche privilégie la simplicité : trois tables, pas de couches inutiles.

CRITÈRES DE SUCCÈS

Critère	Cible
Saisie complète d'une séance (tirage → clôture)	< 2 min
Fonctionnement hors ligne	100 % des fonctions
Perte de données après réinstallation	0 (via backup)
Ouverture à froid de l'app	< 2 s

2. Périmètre

DANS LE PÉRIMÈTRE (V1)

- Gestion des joueurs (création, édition, archivage).
- Création de séance, tirage aléatoire des équipes, édition manuelle.
- Saisie du score, des buts, passes et cartons.
- Clôture de séance avec règles de validation.
- Classement (points totaux + points/match) et statistiques individuelles.
- Statistiques globales du groupe.
- Backup JSON vers Supabase Storage + restauration.

HORS PÉRIMÈTRE (EXCLU)

- Multi-utilisateurs, authentification, rôles, partage.
- Synchronisation bidirectionnelle, résolution de conflits.
- Backend applicatif, API serveur.
- Notifications, invitations, calendrier.
- Tournois, championnats, poules.
- Export PDF/CSV, graphiques avancés.
- `BalancedSplitter` (prévu V2).

3. Stack technique

Couche	Choix technologique
Framework	Flutter (Dart)
UI	Widgets Flutter natifs (implémentation d'un design system unifié, modes clair/sombre basés sur Figma)
Base locale	SQLite via `sqflite` — **Source de vérité**
Packaging	Build APK Android natif
Backup	Supabase Storage via `supabase_flutter` (bucket privé, RLS owner)
Backend	Aucun

CONTRAINTES D'ARCHITECTURE

- Les agrégats ne sont **jamais** calculés côté applicatif (Dart) : ils sortent directement des vues SQL (`v_player_stats`, `v_standings`, `v_group_totals`, `v_session_summary`).
- Une seule connexion SQLite est ouverte au démarrage et fermée à la mise en arrière-plan.
- Les migrations sont gérées par des fichiers SQL numérotés (`001_init.sql`, `002_views.sql`, …) appliqués séquentiellement selon le marqueur `meta.schema_version`.

4. Modèle de données — figé

4.1 TABLES

players : `id` (TEXT, PK), `name` (NOT NULL), `nickname`, `avatar_url`, `is_active` (0/1, def. 1), `created_at`, `updated_at`

sessions : `id` (TEXT, PK), `played_at` (NOT NULL), `status` ∈ { `DRAFT`, `CLOSED` }, `split_mode` ∈ { `RANDOM`, `RANDOM_ADJUSTED`, `MANUAL` }, `allow_uneven_teams` (0/1), `is_ranked` (0/1), `score_a`, `score_b`, `result` ∈ { `A`, `B`, `DRAW` }, `unassigned_goals_a`, `unassigned_goals_b`, `notes`

participations : `id` (TEXT, PK), `session_id` (FK), `player_id` (FK), `team` ∈ { `A`, `B` }, `goals`, `assists`, `yellow_cards`, `red_cards`, **UNIQUE (session_id, player_id)**

meta : `key` (PK), `value` — contient `schema_version`.

4.2 CONTRAINTES GARANTIES PAR LE SCHÉMA

- Une séance `CLOSED` porte obligatoirement un score et un résultat cohérent.
- Un joueur ne peut pas apparaître deux fois dans la même séance (UNIQUE).
- `ON DELETE RESTRICT` : Un joueur ayant des participations ne peut pas être supprimé.
- `ON DELETE CASCADE` : Supprimer une séance supprime ses participations.

5. Règles métier invariantes

> **Attention :** Toute proposition contredisant ces règles lors du développement doit être signalée explicitement, et jamais appliquée silencieusement.

- **Source de vérité unique :** La table `participations` est la seule source. Aucun cumul n'est stocké ; tout est dérivé à la lecture.
- **Barème :** Une séance = un match. 3 pts (victoire), 1 pt (nul), 0 pt (défaite).
- **Clôture bloquante :** Score renseigné, tous les présents affectés, aucun joueur en double, écart d'effectif ≤ 1 (sauf si `allow_uneven_teams = true`).
- **Clôture non bloquante :** Un avertissement est affiché si la somme des buts des joueurs + non attribués diffère du score, mais la clôture reste possible.
- **Séances classées :** Les agrégats de points se basent sur `status = 'CLOSED' AND is_ranked = true`. Les statistiques (buts, passes) comptent même si la séance est amicale.
- **Archivage :** Aucune suppression de joueur ayant joué (`is_active = false`). Il reste dans l'historique.
- **Format :** Le format (ex: 5v5) n'est pas stocké, il est dérivé du comptage par équipe.

6. Périmètre Fonctionnel et Écrans

6.1 GESTION DES JOUEURS

Liste triée par nom (actifs d'abord, archivés repliés). Recherche par nom/surnom. La fiche d'un joueur contient une synthèse de ses performances (points, % victoires), ses statistiques brutes et l'historique de ses matchs.

6.2 GESTION DES SÉANCES

Création rapide avec présélection des joueurs actifs. Génération des équipes via un `RandomSplitter` (Fisher-Yates) produisant un brouillon. Édition manuelle possible (qui écrase le mode aléatoire). Saisie des statistiques individuelles et des buts non attribués avant la validation et la clôture.

6.3 CLASSEMENTS ET STATISTIQUES GLOBALES

Classement principal par assiduité et points totaux. Classement alternatif par ratio (points/match) réservé aux joueurs ayant disputé au moins 50% des rencontres (dénominateur mobile). Vue statistique globale affichant les totaux du groupe (buts, passes, cartons, moyenne).

6.4 RÉGLAGES, BACKUP ET RESTAURATION

Déclenchement d'un snapshot JSON automatisé après chaque clôture vers Supabase. En cas de perte de l'appareil, l'utilisateur peut télécharger la sauvegarde. La restauration implique un *wipe local complet* avant réimport (aucune fusion).

7. Exigences non fonctionnelles

- **Hors ligne :** Toutes les fonctionnalités (hors cloud backup) doivent fonctionner à 100% sans réseau.
- **Performance :** Listes fluides jusqu'à 50 joueurs et 500 séances. Les requêtes via `sqflite` doivent répondre quasi-instantanément.
- **Intégrité :** L'instruction `PRAGMA foreign_keys = ON` doit être exécutée à chaque ouverture de la base.
- **Transactions :** Toute opération multi-tables (ex: clôture, restauration) doit être encapsulée dans une transaction SQL.
- **Ergonomie :** Interface pensée pour un usage à une main sur le bord d'un terrain (cibles tactiles ≥ 44 px).
- **Langue :** Interface utilisateur en français. Code, identifiants et commentaires de code en anglais.

8. Conventions de développement (Flutter)

- Échanges de conception en français, mais **commits (Conventional Commits) et code en anglais**.
- Pas de sur-ingénierie : architecture simple, pas de DDD ni de CQRS complexes.
- L'unique abstraction assumée est l'interface `TeamSplitter` en prévision d'algorithmes plus complexes en V2.

ARBORESCENCE INDICATIVE

```
      lib/ main.dart screens/ players_screen, sessions_screen, standings_screen...
widgets/ player_card, session_card, theme_data models/ (Mapping minimal des retours
SQL) services/ db/ connection, migrations, queries splitter/ TeamSplitter.dart,
RandomSplitter.dart, TeamAssignment.dart backup/ exporter, importer, supabase_client
assets/ sql/migrations/ 001_init.sql, 002_views.sql
```

9. Plan de livraison

Lot	Contenu	Résultat attendu
L1	Initialisation Flutter, connexion sqflite, migrations, CRUD joueurs	Application basique qui liste et crée des joueurs localement
L2	Séances : création, sélection des présents, tirage aléatoire, édition manuelle	Brouillon de composition d'équipes fonctionnel
L3	Saisie des statistiques en match + clôture + règles de validation	Séance enregistrée et close en base de données
L4	Génération des vues SQL pour le classement et les statistiques	Affichage des classements complets et fonctionnels
L5	Implémentation du backup et restauration via le SDK Supabase Flutter	Sauvegarde automatique réussie après une clôture
L6	Build APK natif, polish UI, tests complets sur appareil Android	APK final installable et prêt à l'emploi

10. Risques identifiés

- **Perte de l'appareil local :** Mitigé par le système de backup automatisé après chaque validation de séance.
- **Seuil d'éligibilité instable :** En début de cycle, le seuil de 50% de présence varie beaucoup. Le classement par points bruts restera l'affichage par défaut.
- **Incohérence des scores :** Une alerte non bloquante est générée si la somme des buts individuels ne correspond pas au score global saisi, avec recours aux "buts non attribués".
- **Migration future bloquée :** Refus strict de restaurer un backup dont la version (`schema_version`) est supérieure à celle supportée par l'application locale.

11. Glossaire

Séance : Une soirée de football équivalant à un match unique avec un résultat.

Séance classée : Rencontre comptabilisée pour les points globaux (`is_ranked = 1`). Dans le cas contraire, elle est dite "amicale".

Format : La configuration de la partie (ex: 5v5, 4v5), déduite dynamiquement et jamais stockée en dur.

Buts non attribués : Buts réels (CSC, erreurs d'affectation) ajoutés manuellement pour faire correspondre le total individuel au score final de l'équipe.

Éligible : Statut d'un joueur ayant disputé au moins 50 % des séances classées, lui permettant de figurer au classement par ratio.